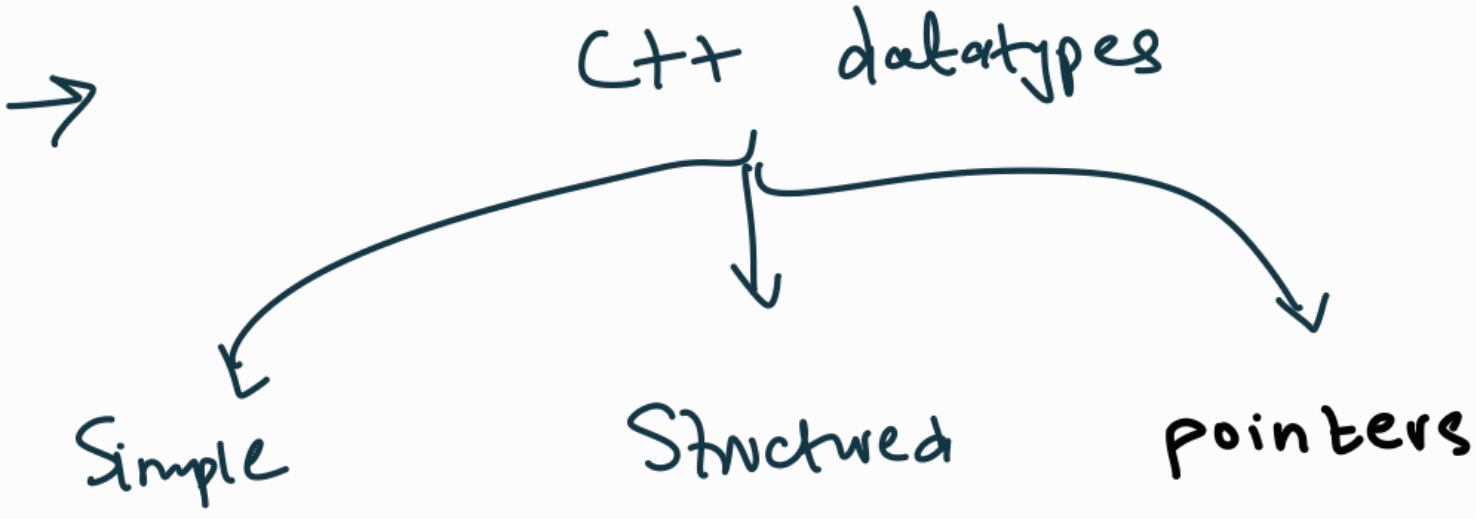
→ Any thing in a double quote is a String.

→ Pressing the Space button automatically adds a Space bc

→ Appendix A → every datatype

→



three catogories of Simple datatype

1) Integral:: deals with integers

2) floating point:: deals with decimal numbers

3) Enumeration:. User-defined data

Char data type → Stores a Single character.

But actually Char can also store Integers b/w -128 ⇔ 127 (ASCII) that represent characters.

Char example = 'A' ;

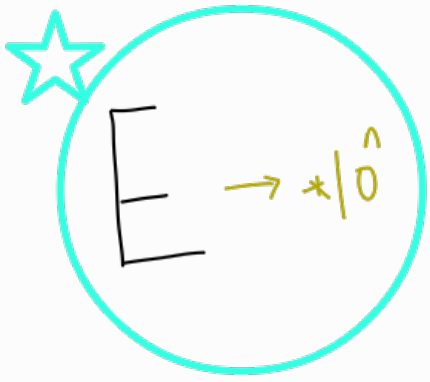
output → A

Char example1 = 65 ;

output → A

Single characters ' '

String / multiple characters " "



for e.g
7.599×10^{-8} in reality
7.599E-8 in cpp

floating point

75.299

Reality

7.5968900E1

in cpp

7.5968900 x 10^1

E → represents exponent along with *10 base.

float memory allocated → 4 bytes

Double memory allocated

→ 8 bytes // more precision

Int also allocates 4 bytes.

% modulus is only used for integer type data. /to find the remainder.

Note: The range of possible remainders is always from 0 ↔ (divisor-1)

e.g.

10 % 3 → Range of remainders 0 ↔ 2 (Ans: 1)

$$3\overline{)10}\ 3$$
$$-\ 9$$
$$1$$

1) $-34 \% 5$

2) $34 \% -5$

3) $-34 \% -5$

ORDER of Precedence

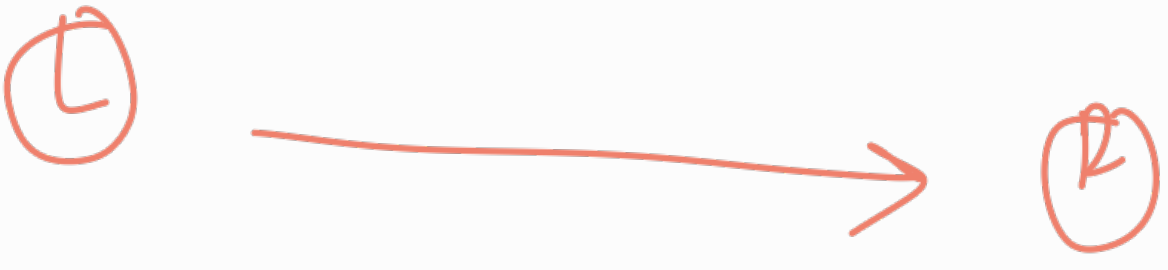
* / %

+ −

← Same level of precedence

When Same level of precedence Calculations are performed from Left → Right

3 * 7 − 6 + 2 * 5 / 4 + 6



But ( ) Brackets is ~~highest in order of~~ Precedence

highest precedence is for unary operators.

$3+4*5 = 23$

$(3+4)*5 = 35$

Note:: All the letters in a key board are CHAR (data type) each having there own Ascii values.

Char '8' = 56 (Ascii)

int 8 = 8

Integer value of '8' is

56

'8' + 7

= 56 + 7 = 63

'8' + '7'

= 56 + 55 = 111

Note if we have both

Int + float datatype

Answer is given in

float

$5.4 * 2 - 13.6 + 18 / 2$

$10.8 - 13.6 + 9 = 6.2$

$15.6 \; 12 + 5$

$= 7.8 + 5.0 = \underline{12.8}$

$4 + 5 \; 12.0$

$= 4 + 2.5 = 6.5$

$4 * 3 + 7 \; 15 - 25.5$

$12.0 + 1 - 25.5 = -12.5$

7/5 = 5 1
7
5
2



Type-casting:: Convert a variable from one datatype to another.

we can convert a double/float to int but not viceversa as int is decimal part is removed.

So you can only convert from a Big datatype

to small due to memory allocation. But not from small to big datatype

```
double b3 = 12.3265789 ;
float c2 = (float) b3;
cout<< c2;
```

+c

Output = 12.3578.

Another method

Static_cast < target data type > (expression to be evaluated)



Static_cast <Dty> (expression)

String. (Datatype requires a library)

↳ enclosed in double Quotations.

"" → empty string.

"William Jacab"] e.g.s

length of a string is no of characters in a string.

0 1 2 3 4 5 6 7 8 9 10 11 12
William Jacab → position starting from Zero.

Max position :: 12 ←

length :: 13 ←

to make sure a variable stays constant use.

Const <datatype> name = value;
given

e.g

const int b = 6; // Value remains unchanged

To represent blank space in String we use " " and for Char we use ' '.

A datatype is set to be Simple if it stores One value at a given time

(Increment/decrement operators:)

Pre decrement	Post decrement
-- Variable	Variable --
// Increment	// Increment
++ Variable	Variable--

++x means first the value of x is incremented and then this new value is used in a given expression.

---

x++ means first the expression is evaluated (or value assigned) then the value of x is incremented.

y=

```
x=5;
y=++x;
```

```
x=6;
y=6;
```

```
x=5;
y=x++;
```

```
x=6;
y=5
```

\t → Creates a horizontal Space tab.

\' → adds Single quotation marks.

\" → adds double quotation marks

#Include <headerfile name>

↓

Expands the functionalities of your program.

//Appendix F shows common header files//

(#Include <String> for String datatype)

Ch:3 Basic functions of a program

(get data from user → Manipulate → output)

I/O

Input Stream

Output Stream

→ Brings series of data (stream) into the program

→ Sends data out of program.

Datatype
istream

Datatype
ostream

e.g (get, ignore, and peek putback)

note:: functions are sub programs that are inside of a header file

pow. (x,y) :- x^y

sqrt Squareroots the parameter.

Note

cin>>

↳ extraction operator skips blanck spaces / spaces.

cout<< → insertion operator does not skip spaces.

```
cin >> ch1>> ch2>> num;
```

Input data

A
B
25

Input data

25 → ch2
ch1 A
B

No more valid data type for int

hence

output

2 5 0

A B 25

```
cin >> ch1>> ch2 >> num
```

first input gets in ch1

Second input

third input

to process the entire input also including spaces / \n characters

cin.get (varchar);

used to read character data

cin.get (varchar);

used only for variable with a char datatype

String name; cin >> name;

Input

Alice Wonderland

Space

output Alice (^^)

to read and input an entire String we use

getline(cin, str var);

String variable

getline(cin, name);

(file input | output)

use header file (fstream)

file input → viewing the contents of a file | opening it
(Reading data)

file output → Saving | Sending info to a file
(Writing data)

fstream header file contains two data types



ifstream

ofstream

(Input file Stream)

(output file Stream)

five step process

1) Include header file fstream in the program

2) Declare variables (file stream)

3) Associate file stream variables with input/output/opeing files.

4) Use file stream variables with >> <<. to do calculations from file data.

5) Close the files.

Stream variables e.g

ifstream Inputdata ;

ofstream OUtputdata ;

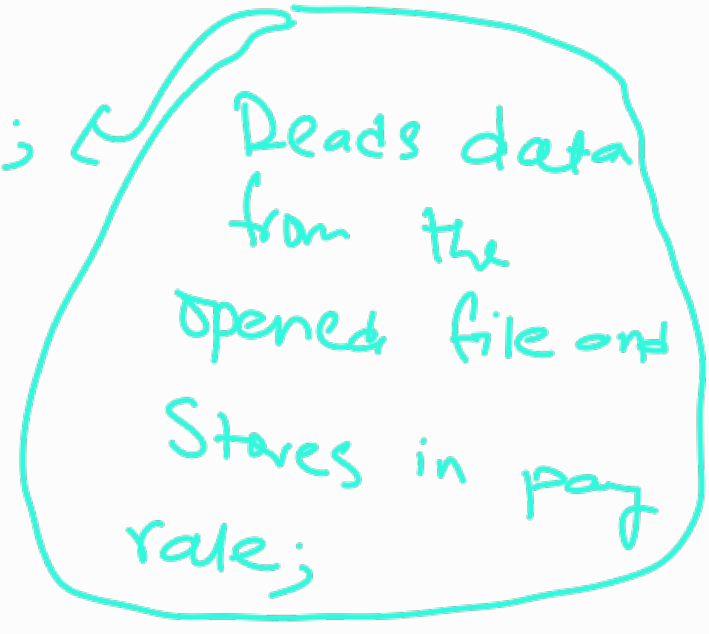
Opening a file Syntax.

filestream Variable . open (Source Name);

where Sourcename is filename

Now to do Calculations with the file data you simply use >> for inputting data from the file you just opened but using the file stream variable.

Input data >> pay rate;



Output data << "The pay check is " << pay;

outputs from the fileStream variable using the file.

filestream Variables >> (for input)

filestream Variables << (for output)

last step is close the files

```
filestreamVariables.close();
```

for opening.

```
filestreamVariables.open("file name")
```

Make Sure to open and close file both for Reading (Input) and writing (Output)

Note we can either open a file for Input or for output one at a time.

Outdata << alpha



data content writing to filestream variable then to file

Indata >> alpha



Reading data from the file and storing it in a variable called alpha.

(Control Statements)

In CPP a condition is expressed by logical (Boolean) expression

(i > J) if i > J then expression is true. otherwise it will be false

for char the expression evaluates according to the ASCII values

'R' > 'T' is false

ASCII 82 > 84

'A' <= 'a' is true

(65 <= 97)

Note: that Comparing two different datatypes produces unexpected Results.

8 < '5'

8 < 53 → true

True → Integer value (1)

false → Integer value (0)

>= greater than OR equal to

<= less than OR equal to

(String comparision)

Are compared from the Starting to character by character UNTIL a mismatch is found or compares till the last characters if they are equal.

e.g

"Hello" < "Hi"

H ⟷ H

e ⟷ i

e < i

true

"Hello" > "Hen"

H ⟷ H

e ⟷ e

L ⟷ n

L > n

(false)

! → Not reverses operations.

!(6 <= 7) → false

Note for || (OR) operator any of 1 expression must be true inorder to give result true.

Order of precedence

- ()
- ! + - (unary op) (highest)
- * / %
- + -
- < <= > =>
- == !=

- &&
- ||
- = (assignment)

(lowest)

```
if (expression 1)
    Statement 1
else
    Statement 2
```

if expression 1 is true

Only Statement 1 executes.

if expression 1 is false then Only Statement 2 executes.

Note

```
if (score >= 60)
cout<< "passing";
cout<< "failing";
```

if the expression fails the output would immediately be the next line!!!.

However if expression is true it will

Output both passing and failing.

for using absolute value function use

`fabs(b-a)`

Remember char uses 1 byte and Int uses 4 bytes

```
if ( )
{
    if ()
    {
        if ()
```

nested ifs

```
if ()
{
    if ()
    {
    }
    else
    {
    }
}
else
{
    if ()
    {
    }
    else {
```



nested if else

nested if's are used when conditions are dependant on each other!

```
if (go to Supermarket)
    {
        if (apples available)
            {
                buy;
            }
    }
```

Or in simpler terms when you need to make multiple decisions within a decision!

Syntax of nested if.

```
if (condition)
{
    if (condition)
    {
    }
}
```

Nested if else

```
if (condition)
{
    if (condition)
    {
    }
    else
    {
    }
}
```

else

\t → Creates a horizontal Space tab.

\' → adds Single quotation marks.

\" → adds double quotation marks

\\ → adds a bACKslash \

Switch case → multiple choice just like if Statements.

Syntax

```
Switch (expression)
  {
      case 1: Statement;
              break;
      case2: Statement;
             break;
```

```
    ⋮
  case n: Statement;
          break;
  default: Statement;
}
```

Runs only if all other cases are false